

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CSE 210 JAN 2026 SEMESTER
COMPUTER ARCHITECTURE SESSIONAL

Assignment on 4-bit MIPS Design and Simulation

In this assignment, you will design and simulate (in S/W) a modified and reduced version of the MIPS instruction set.

1 DESIGN SPECIFICATION

- The Address bus will be 8 bits.
- The Data bus will be 4 bits.
- You have to implement a 4-bit ALU, hence the name 4-bit MIPS.
- You have to design the following temporary registers:

\$zero, \$t0, \$t1, \$t2, \$t3, \$t4

All these registers will be 4 bits, and the assembly code provided to simulate your design will use only the mentioned registers.

- The Control unit should be microprogrammed. The control signals associated with the operations should be stored in a special memory (you can use a separate ROM for this purpose) in units called Control Words.
- During the simulation, an Assembly Code will be given, and you have to convert it into your designed MIPS machine code. The conversion process should be automatic. For this conversion, you can write code in your preferred programming language.
- Marks will vary according to the efficiency of the design.

2 INSTRUCTION SET DESCRIPTION

Instruction ID	Instruction Type	Instruction
A	Arithmetic	add
B	Arithmetic	addi
C	Arithmetic	sub
D	Arithmetic	subi
E	Logic	and
F	Logic	andi
G	Logic	or
H	Logic	ori
I	Logic	sll
J	Logic	srl
K	Logic	nor
L	Memory	lw
M	Memory	sw
N	Control	beq
O	Control	bneq
P	Control	j

4
2 = 16

0000
1111

3 MIPS INSTRUCTION FORMAT

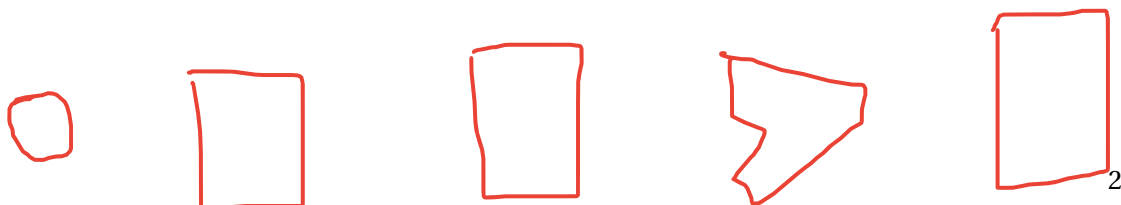
Our MIPS Instructions will be 16 bits long with the following four formats.

• R-type	<table><tr><td>Opcode</td><td>Src Reg 1</td><td>Src Reg 2</td><td>Dst Reg</td></tr><tr><td>4-bits</td><td>4-bits</td><td>4-bits</td><td>4-bits</td></tr></table>	Opcode	Src Reg 1	Src Reg 2	Dst Reg	4-bits	4-bits	4-bits	4-bits
Opcode	Src Reg 1	Src Reg 2	Dst Reg						
4-bits	4-bits	4-bits	4-bits						
• S-type	<table><tr><td>Opcode</td><td>Src Reg 1</td><td>Dst Reg</td><td>Shamt</td></tr><tr><td>4-bits</td><td>4-bits</td><td>4-bits</td><td>4-bits</td></tr></table>	Opcode	Src Reg 1	Dst Reg	Shamt	4-bits	4-bits	4-bits	4-bits
Opcode	Src Reg 1	Dst Reg	Shamt						
4-bits	4-bits	4-bits	4-bits						
• I-type	<table><tr><td>Opcode</td><td>Src Reg 1</td><td>Src Reg 2/Dst Reg</td><td>Addr./Immdt.</td></tr><tr><td>4-bits</td><td>4-bits</td><td>4-bits</td><td>4-bits</td></tr></table>	Opcode	Src Reg 1	Src Reg 2/Dst Reg	Addr./Immdt.	4-bits	4-bits	4-bits	4-bits
Opcode	Src Reg 1	Src Reg 2/Dst Reg	Addr./Immdt.						
4-bits	4-bits	4-bits	4-bits						
• J-type	<table><tr><td>Opcode</td><td colspan="2">Target Jump Address</td><td>0</td></tr><tr><td>4-bits</td><td colspan="2">8-bits</td><td>4-bits</td></tr></table>	Opcode	Target Jump Address		0	4-bits	8-bits		4-bits
Opcode	Target Jump Address		0						
4-bits	8-bits		4-bits						

4. MEMORY CONSIDERATIONS

You need to consider three types of memory:

- Instruction Memory (accessed through the program counter, PC)
- Data Memory (accessed through address)
- Bonus: Stack Memory (accessed through stack pointer, sp. Sample instruction: **sw \$t0, 0(\$sp)** or **lw \$t1, 4(\$sp)**), this will be used for push and pop instructions.



5 INSTRUCTION SET ASSIGNMENT

The opcodes of the instruction will be between 0 to 15 based on the sequence of instruction id given below. Sequence **ABCDEFGHIJKLMN** means add instruction's opcode will be 0, addi instruction's opcode will be 1, sub instruction's opcode will be 2, and so on.

Group ID	Section A1	Section A2	Section B1	Section B2
1	IFMJHLNGDABECOKP	BDNKMFIJALIPHGOEC	PDKLOFCHBAIMNEGJ	NJDFPGLABKMEOHCI
2	MIOADKFHPCBJENLG	JOCMGNDHFIELKBAP	AJGBFIKMPNODELCH	EJDPMLKACIHONGFB
3	NHDIAJEMKBOCFPLG	NCDIBGHJKAPOMLEF	OJNCAEPLGKMFDDBHI	FDIMOKLNGHEPJCB
4	GHPDKBANILFCMOJE	GCKEOPDAMHINFLBJ	AGMKBPONCIDHLEFJE	DFJGAEMKNOBCHLIP
5	JLCDHEAKMFNGIBPO	KNOHAEBFIJGLPCDM	IACKOLEMPBJDNFHG	CPGILOMBEADFNJHK
6	DKCALHMFJGOIPBNE	BEMJLDGAHOICKNPF	FOPNAKDHBLCIEMGJ	KHODFAJIENBGPCLM

Group ID	Section C1	Section C2
1	EFHIJMNODBPAKLCG	IDBFMPAGKJNOLCHE
2	ANJBFPIHGEMODLCK	KAJMGJLNFIEBDOCHP
3	GKJNAFLBEHPOMCID	KEHLPJIFBAGODNMC
4	DOPBLGIFAJMEKNCH	BDFHMCOLNPGJKAIE
5	JEBONAMIFGPCKHDL	HJDCMKENFILPABGO
6	OACKHIFDNBJPMLGE	EMAFLKIBHCDGPJON

6 REPORT CONTENT

The contents of the report are recommended as follows:

- Introduction
- Instruction Set
- Circuit Diagram printed on drawing paper
- How to write and execute a program on this machine
- Special Features Implemented, if any (may carry bonus marks)
- ICs used with their count
- Discussion

7 SUBMISSION GUIDELINE

- Create a folder named "<Lab Group>_<Group ID>_Simulation" and put all the necessary simulation files in this folder.
- Create a second folder named "<Lab Group>_<Group ID>_Necessary Content" and put all your codes to convert assembly codes into MIPS machine codes (or any other extra contents that will be necessary to simulate you pc).

- Finally, create a third folder named "<Lab Group>_<Group ID>_Submission" and put your report along with the previously prepared two folders. Now zip this final folder and upload the zip file to the Moodle submission link (single submission from each group).
- Remember that proper submission will carry 10% of total assignment marks.
- **Submission Deadline: September 19 (Saturday), 2 PM.**

N.B. This document may be updated if needed. Notifications will be given in the news forum as soon as any corrections are made.

For any kind of confusion, feel free to email at rifat@cse.buet.ac.bd with the subject "Jan 2026 CSE 210 Assignment 3".